

TP Master IA

Comparaison d'algorithmes de classification et de régression

1. Objectif du TP

L'objectif de ce TP est de :

- Mettre en œuvre plusieurs algorithmes d'apprentissage supervisé
- Comparer leurs performances sur un même jeu de données
- Comprendre l'impact du prétraitement des données
- Développer une capacité d'analyse critique des résultats

2. Travail demandé

2.1 Compréhension des données

Choisir un dataset.

À faire :

- Identifier :
 - Variable cible (target)
 - Variables explicatives (features)
- Déterminer les types de variables :
 - Numériques (discrètes / continues)
 - Catégorielles (nominales / ordinales)
- Analyse exploratoire :
 - Statistiques descriptives (moyenne, médiane, etc.)
 - Visualisations :
 - Histogrammes
 - Boxplots
 - Corrélations

☞ Objectif : comprendre la structure des données avant la modélisation.

2.2 Prétraitement des données

a) Gestion des valeurs manquantes

- Suppression des lignes
- Imputation :
 - Moyenne / médiane

- Mode
- KNN Imputer (option avancée)

b) Encodage des variables catégorielles

- Label Encoding
- One-Hot Encoding

c) Normalisation / Standardisation

- Min-Max Scaling
- StandardScaler (centrage-réduction)

☞ Important pour :

- KNN
- SVM
- Régression

d) Traitement des valeurs aberrantes (outliers)

- Méthodes :
 - Z-score
 - IQR (Interquartile Range)
- Suppression ou transformation

2.3 Modèles à implémenter

◆ Classification (si dataset de classification)

- Régression logistique
- KNN (K-Nearest Neighbors)
- Arbre de décision
- Forêt aléatoire
- SVM (Support Vector Machine)
- XGBoost (option avancée)

◆ Régression (si dataset de régression)

- Régression linéaire
- SVR (Support Vector Regression)
- Arbre de régression
- Random Forest Regressor
- XGBoost Regressor

2.4 Évaluation des modèles

◆ Classification

- Accuracy
- Precision
- Recall
- F1-score
- ROC AUC
- Matrice de confusion

☞ Choisir au moins 2 métriques avancées supplémentaires et justifier leur choix.

Métriques avancées

- Balanced Accuracy (utile si classes déséquilibrées)
- Matthews Correlation Coefficient (MCC) (très robuste)
- Cohen's Kappa (mesure d'accord corrigé du hasard)

Métriques probabilistes

- Log Loss (Cross-Entropy)
- Brier Score (qualité des probabilités prédites)

Autres courbes utiles

- PR AUC (Precision-Recall AUC)
☞ plus pertinent que ROC AUC si dataset déséquilibré

☞ Interprétation attendue :

- Analyse des erreurs (FP/FN) et de leur impact
- Étude du déséquilibre des classes et choix des métriques adaptées
- Interprétation de la matrice de confusion

◆ Régression

- MSE (Mean Squared Error)
- RMSE
- MAE (Mean Absolute Error)
- R^2 (coefficient de détermination)

☞ Choisir au moins 2 métriques avancées supplémentaires et justifier leur choix.

Métriques avancées

Métriques supplémentaires

- MAPE (Mean Absolute Percentage Error)
- MedAE (Median Absolute Error) (robuste aux outliers)

Métriques avancées

- RMSLE (Root Mean Squared Log Error)
- Explained Variance Score

🔗 Interprétation :

- Analyse de l'erreur (ampleur, distribution, sensibilité aux outliers)
- Évaluation de la qualité globale des prédictions (R^2 , tendance)
- Étude des résidus pour valider la pertinence du modèle

2.5 Analyse des résultats

À faire :

- Comparer les performances des modèles
- Identifier :
 - Meilleur modèle
 - Modèle le plus stable
- Analyser :
 - Overfitting / Underfitting
- Impact du prétraitement

Questions à traiter :

- Pourquoi un modèle est-il meilleur qu'un autre ?
- Quel modèle est le plus robuste ?
- L'augmentation des données améliore-t-elle les résultats ?
- Le scaling a-t-il influencé les performances ?
 - Comparer les performances avec et sans scaling
 - Observer les changements sur les métriques (Accuracy, F1, RMSE, etc.)
 - Expliquer pourquoi certains modèles sont affectés et d'autres non

3. Implémentation (exemple Python – scikit-learn)

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score
from sklearn.neighbors import KNeighborsClassifier
```

```
# Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Normalisation
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Modèle
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)

# Prédiction
y_pred = model.predict(X_test)

# Evaluation
print("Accuracy:", accuracy_score(y_test, y_pred))
```

4. Structure du rapport (Article scientifique)

◆ Abstract

Résumé du travail (objectif + résultats principaux)

◆ Introduction

- Contexte
- Importance du problème

◆ État de l'art

- Présentation des algorithmes utilisés
- Avantages / inconvénients
- Les travaux ayant traité la même idée

◆ Méthodologie

- Dataset
- Prétraitement
- Choix des modèles
- Schéma (pour mieux expliquer cette partie)

◆ Application

- Implémentation
- Outils utilisés (Python, sklearn...)

◆ Résultats & Discussion

- Tableaux comparatifs
- Graphiques
- Analyse critique

◆ Conclusion

- Résumé
- Perspectives

◆ Références

- Articles
- Documentation

5. Contraintes

- Travail individuel
- Justification obligatoire des choix
- L'utilisation d'IA est non permise

6. Bonus avancé (fortement recommandé)

Validation croisée avancée

- Stratified K-Fold (classification déséquilibrée)
- Time Series Split (si données temporelles)
- Nested Cross-Validation (évaluation + tuning sans fuite de données)

Optimisation des hyperparamètres avancée

- Randomized Search (plus efficace que GridSearch sur grands espaces)
- Bayesian Optimization (Optuna / Hyperopt)
- Analyse de sensibilité des hyperparamètres

Interprétabilité des modèles (Explainable AI - XAI)

- SHAP values (impact global et local des variables)

- LIME (explication locale des prédictions)
- Partial Dependence Plots (PDP)

Analyse avancée des performances

- Courbes Precision-Recall (plus pertinentes que ROC en cas de déséquilibre)
- Calibration curves (fiabilité des probabilités prédites)
- Learning curves (détection overfitting / underfitting)

Robustesse et généralisation

- Test sur données bruitées (noise injection)
- Cross-dataset validation (test sur un autre dataset proche)
- Bootstrap evaluation (intervalle de confiance des performances)

Comparaison avancée des modèles

- Analyse bias-variance tradeoff
- Stacked models / ensemble learning (stacking, blending)
- Comparaison avec modèles baselines multiples (naïf, logistique, majorité)

Détection d'instabilité

- Variance des résultats entre folds
- Sensibilité aux outliers
- Stability selection (importance stable des features)